



Department of Computer Science and Engineering

ALGORITHMIC PROBLEM-SOLVING HACKATHON (APSH 2026)

PROJECT DOCUMENTATION

Date:03-10-2026 | Venue:3rdFloor,A&BBlocks

Field	Details
Project Title	Electric Vehicle Charging Station Planner
Team Name / Team No.	06
Year / Semester / Section	II year B.Tech / III sem / B
Algorithm Paradigm Used	Multi-Factor Weighted Scoring, Feasibility Filtering & Ranking
Programming Language / Tools	Python 3, Flask, JavaScript, Leaflet, OpenStreetMap, OSRM
Faculty Mentor	Mrs . V.Tejaswi

Team Members

S.No	Name of the Student	Roll No.	Role
1	Sowmya Yadav	25MVCSER0114	Team Lead
2	Dasari Nithyasri	25MVCSER0110	Documentation & QA Specialist
3	M Nageshwari	25MVCSER0140	frontend lead, UI & Testing lead
4	P.Genelia	25MVCSER0155	Backend API

1. Problem Statement

Electric vehicle drivers need to choose a charging station using multiple practical factors instead of simply selecting the closest station. A nearby station may be full, may offer a slower charger, may require more charging time, or may not be safely reachable with the vehicle's current battery level.

Inputs: Vehicle ID, current latitude/longitude and battery percentage; charging-station ID, location, available slots, total slots, charger type, charging speed, estimated charging time and guard availability.

Outputs: A ranked list of feasible charging stations, the recommended highest-scoring station, score breakdown/reasons, map navigation information and battery/charging status.

Constraints: A station is feasible only when it has an available slot and the EV has enough battery to reach it with the defined safety margin. Charging is enabled after the simulated vehicle reaches the selected station.

2. Objectives

- Recommend a suitable charging station using multiple weighted factors rather than distance alone.
- Check battery feasibility and provide low/critical battery warnings.
- Provide an interactive map, route visualization and exact-spot vehicle movement.
- Simulate charging, guard notification, mobile OTP login and emergency handling at 0% battery.
- Demonstrate DAA concepts through distance calculation, candidate evaluation, ranking and complexity analysis.

3. Proposed Approach

Stage1 — Distance and Feasibility Evaluation: For every charging station, the system calculates geographic distance using the Haversine formula. It estimates the battery percentage required to reach the station and checks the 5% safety margin and slot availability.

Stage 2 — Multi-Factor Weighted Scoring: Each station receives scores for distance, availability, charging time, battery safety, charger type and guard availability. The total score is used to compare feasible candidates.

Stage 3 — Ranking and Recommendation: Feasible candidates are ranked in descending order of total score. If no feasible candidate exists, the system falls back to available stations so that alternatives can still be displayed.

Stage 4 — Navigation and Charging: The selected station is displayed on the Leaflet map. The backend can request a road route through OSRM. In the prototype, the EV then moves toward the exact station coordinates and charging becomes available after arrival.

EV Location + Battery	→	Station Evaluation	→	Feasibility Check
Feasible Stations	→	Weighted Score	→	Ranking
Best Candidate		Navigation		Arrival & Charging

Alternative considered: Selecting only the nearest station. **Reason for rejection:** proximity alone ignores slot availability, battery safety, charger type and charging time.

4. Algorithm / Pseudocode

```
FUNCTION RecommendStation(Vehicle, Stations):
  Evaluated = []
  FOR EACH station IN Stations:
    distance = Haversine(Vehicle.location, station.location)
    requiredBattery = distance × 0.20 / 50 × 100
    feasible = (Vehicle.battery ≥ requiredBattery + 5) AND (station.available_slots > 0)
    score = DistanceScore(distance) + AvailabilityScore(station) + ChargingTimeScore(station) +
    BatterySafetyScore(Vehicle, distance) + ChargerTypeScore(station) + GuardScore(station)
    APPEND (station, distance, score, feasible) TO Evaluated
  Candidates = feasible stations
  IF Candidates is empty:
    Candidates = available stations
  SORT Candidates BY score DESCENDING
  RETURN Candidates[0], Evaluated
```

5. Complexity Analysis

Complexity	Best Case	Average Case	Worst Case
Time	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
Space	$O(n)$	$O(n)$	$O(n)$

Justification:

Haversine distance calculation takes **$O(1)$** time for one station because it uses a fixed number of mathematical operations.

For **n** stations, evaluating all stations takes **$O(n)$** . The current implementation sorts the candidate stations by score, giving **$O(n \log n)$** ranking time.

The evaluated station records and ranking lists require **$O(n)$** additional space. External reverse-geocoding and routing network calls are not included in the DAA complexity because their running time depends on external services.

If only the top **k** stations are required, a priority-queue/heap approach can be used with approximately **$O(n \log k)$** ranking time. The current implementation uses complete sorting.

6. Implementation

CoreModules:app.py contains the Flask server, station data, Haversine distance function, station evaluation/ranking engine, OTP flow, reverse geocoding, routing endpoint, nearest-station emergency logic and guard notification simulation.

Frontend: index.html, login.html, app.js and style.css provide the dashboard, interactive map, battery controls, station comparison, navigation, charging simulation and emergency interface.

Data Structures Used: Python lists for station collections and evaluated candidates; dictionaries for vehicle and station records; sorting for complete ranking; optional heap-based top-k ranking.

Tools & Platform: Python 3, Flask, JavaScript, Leaflet, OpenStreetMap/Nominatim, OSRM routing service and browser Geolocation API.

6. Implementation Details

Component	Implementation
Distance	Haversine formula using latitude/longitude
Station evaluation	Distance + availability + charging time + battery safety + charger type + guard
Map	Leaflet with OpenStreetMap tiles
Reverse geocoding	OpenStreetMap Nominatim
Road routing	OSRM driving route API
Authentication	Mobile number + 6-digit OTP + Flask session
Low-battery alerts	Simulated mobile alerts; provider-ready Twilio integration
Emergency	0% battery workflow with nearest-station lookup
Charging	Arrival-gated charging simulation to 100%

Important prototype behavior: At battery $\leq 20\%$, a low-battery warning is shown. At $\leq 10\%$, the warning becomes critical. At exactly 0%, the vehicle simulation stops and the emergency workflow opens.

SMS limitation: The application deliberately labels phone alerts as simulated when no SMS provider credentials are configured. A real provider can be connected through the existing provider-ready function.

Navigation limitation: The OSRM route is real road-routing data when the service is reachable, while the vehicle movement in the college demonstration is simulated.

7. Results and Test Cases

Test Case	Input / Action	Expected Result
TC01	Battery 72%, vehicle near HITEC City	Rank available stations; HITEC City Fast Charge receives the highest score in the def
TC02	Select a full station	Station is penalized and is not included in the feasible recommendation list.
TC03	Low battery $\leq 20\%$	Low-battery warning and simulated phone alert are triggered.
TC04	Critical battery $\leq 10\%$	Critical battery warning is displayed.
TC05	Battery = 0%	Vehicle stops and emergency interface identifies the geographically nearest station.
TC06	Go to Exact Spot	Vehicle moves to selected station coordinates and remains there after arrival.
TC07	Start Charging before arrival	Charging is blocked until the vehicle reaches the selected station.
TC08	Start Charging after arrival	Battery increases through the charging simulation until 100%.

Default demo observation: with the prototype vehicle at approximately 17.4445, 78.3772 and 72% battery, the current station data produces representative scores: HITEC City Fast Charge ≈ 98.64 , Madhapur EV Hub ≈ 91.95 , Financial District Charge Park ≈ 83.36 , Gachibowli Green Charge ≈ 71.83 , while the full Kondapur Power Point is strongly penalized. These values are produced by the project's current scoring weights.

Performance Comparison

Approach	Behavior	Complexity / Observation
Nearest-only selection	Chooses minimum geographic distance	Does not consider slot, battery safety, charger type or ch
Proposed weighted ranking	Filters feasible candidates and scores multiple factors	$O(n)$ evaluation + $O(n \log n)$ sorting.
Top-k heap variant	Keeps only best k candidates	Approximately $O(n \log k)$ when top-k is sufficient.

8. Conclusion and Future Scope

Conclusion: The Electric Vehicle Charging Station Planner demonstrates how algorithmic problem-solving can be applied to a practical EV charging decision. The system evaluates charging stations using distance, battery feasibility, availability, charging time, charger type and guard availability, then ranks the feasible candidates using a weighted score. The prototype combines the DAA engine with an interactive map, navigation, battery monitoring, charging simulation and emergency handling.

Future Scope:

1. Integrate real-time charging-station APIs and live slot availability.
2. Add traffic-aware route duration and dynamic ETA calculation.
3. Add real station reservation and payment integration.
4. Replace simulated SMS with a production SMS/notification provider.
5. Store users, vehicles, bookings and station data in MySQL/PostgreSQL.
6. Add ML-based charging-demand prediction and dynamic electricity-price optimization.
7. Extend the system to multiple EVs and fleet-level charging planning.

9. References

1. T.H. Cormen, C.E. Leiserson, R. L. Rivest, and C. Stein, Introduction to Algorithms, 4th ed., MIT Press, 2022.
2. Flask Documentation, Pallets Projects, Python web framework documentation.
3. Leaflet Documentation, interactive JavaScript maps.
4. OpenStreetMap / Nominatim documentation for map data and reverse geocoding.
5. OSRM Project documentation for open-source road routing.
6. Python Documentation for sorting and heapq modules.

Presentation note: Replace the bracketed team/mentor/roll-number fields before submitting the document.